Polycopié de cours

Vision par Ordinateur et Réseaux de Neurones Convolutifs

De la théorie du signal aux architectures profondes

Niveau Licence 3 — Informatique / Mathématiques Appliquées

Année universitaire 2025–2026

Table des matières

1 Introduction à la vision par ordinateur **5**
1.1 Qu'est-ce que la vision par ordinateur ? . . . 5
1.2 Historique et évolutions majeures . . . 5
1.3 Tâches fondamentales . . . 6
1.4 Enjeux et défis actuels . . . 6

2 Représentation mathématique des images **7**
2.1 Image numérique : définition formelle . . . 7
2.2 Espaces de couleur . . . 7
2.3 Représentation tensorielle . . . 8
2.4 Histogramme et statistiques d'une image . . . 8
2.5 Notion de voisinage et connectivité . . . 8

3 Convolution discrète **9**
3.1 Définition mathématique . . . 9
3.2 Propriétés fondamentales . . . 9
3.3 Gestion des bords . . . 9
3.4 Stride et dilatation . . . 10
3.5 Complexité computationnelle . . . 10

4 Filtres et extraction de caractéristiques **11**
4.1 Filtres de lissage . . . 11
4.2 Détection de contours . . . 11
4.3 Détection de coins et points d'intérêt . . . 12
4.4 Descripteurs locaux : de SIFT aux features apprises . . . 12

5 Architecture des réseaux de neurones convolutifs **13**
5.1 Le neurone artificiel . . . 13
5.2 Fonctions d'activation . . . 13
5.3 Couche convolutive . . . 14
5.4 Couche de pooling . . . 14
5.5 Architecture canonique d'un CNN . . . 14

6 Optimisation et apprentissage **15**
6.1 Fonction de perte . . . 15
6.2 Descente de gradient stochastique (SGD) . . . 15
6.3 Algorithmes d'optimisation avancés . . . 15
6.3.1 SGD avec momentum . . . 16
6.3.2 Adam . . . 16
6.4 Rétropropagation du gradient . . . 16
6.5 Planification du taux d'apprentissage . . . 16

7 Régularisation **19**
7.1 Le problème du surapprentissage . . . 19
7.2 Régularisation L2 (Weight Decay) . . . 19
7.3 Dropout . . . 19
7.4 Batch Normalization . . . 20
7.5 Augmentation de données . . . 20
7.6 Early Stopping . . . 20

8 Architectures modernes **21**
8.1 VGGNet (2014) . . . 21
8.2 GoogLeNet / Inception (2014) . . . 21
8.3 ResNet (2015) . . . 21
8.4 DenseNet (2017) . . . 22
8.5 EfficientNet (2019) . . . 22
8.6 Vision Transformers (2020) . . . 22

9 Explainable AI (XAI) **25**
9.1 Pourquoi l'explicabilité ? . . . 25
9.2 Taxonomie des méthodes d'explicabilité . . . 25
9.3 LIME (*Local Interpretable Model-agnostic Explanations*) . . . 25
9.4 SHAP (*SHapley Additive exPlanations*) . . . 26
9.5 Méthodes basées sur le gradient . . . 26

10 Grad-CAM **27**
10.1 Principe général . . . 27
10.2 Formulation mathématique . . . 27
10.3 Algorithme détaillé . . . 27
10.4 Variantes . . . 28
10.5 Interprétation et limites . . . 28

11 Implémentation PyTorch **29**
11.1 Introduction à PyTorch . . . 29
11.2 Tenseurs et opérations de base . . . 29
11.3 Définition d'un CNN avec `nn.Module` . . . 29
11.4 Boucle d'entraînement . . . 30
11.5 Chargement de données et augmentation . . . 30
11.6 Transfer learning . . . 31
11.7 Implémentation de Grad-CAM . . . 31

12 Projet : application en imagerie médicale **33**
12.1 Contexte : classification de radiographies thoraciques . . . 33
12.2 Jeu de données . . . 33
12.3 Pipeline complet . . . 33
12.3.1 Prétraitement des données . . . 34
12.3.2 Gestion du déséquilibre de classes . . . 34
12.3.3 Fine-tuning progressif . . . 34
12.4 Métriques d'évaluation . . . 35
12.5 Explicabilité médicale avec Grad-CAM . . . 35
12.6 Considérations éthiques et réglementaires . . . 36
12.7 Cahier des charges du projet . . . 36

Chapitre 1

Introduction à la vision par ordinateur

1.1 Qu'est-ce que la vision par ordinateur ?

La vision par ordinateur (*Computer Vision*) est une branche de l'intelligence artificielle qui vise à donner aux machines la capacité d'interpréter et de comprendre le contenu visuel du monde réel à partir d'images ou de vidéos. Ce domaine se situe à l'intersection de l'informatique, des mathématiques appliquées et des neurosciences.

L'œil humain perçoit la lumière grâce aux photorécepteurs de la rétine (cônes et bâtonnets), puis le cortex visuel interprète ces signaux en une représentation cohérente du monde. Reproduire ce processus par voie algorithmique représente un défi considérable : une image de 1024×768 pixels en couleur contient près de 2,4 millions de valeurs numériques, et la sémantique de la scène n'est absolument pas évidente à partir de ces seules valeurs.

La vision par ordinateur couvre un large spectre de tâches, allant de la simple détection de contours jusqu'à la compréhension sémantique de scènes complexes. Historiquement, les premières approches reposaient sur des méthodes de traitement du signal et de géométrie projective. Depuis les années 2010, l'apprentissage profond a profondément transformé le domaine, offrant des performances parfois supérieures à celles de l'humain sur certaines tâches de classification.

1.2 Historique et évolutions majeures

Le domaine de la vision par ordinateur connaît une longue histoire. Dès les années 1960, Larry Roberts au MIT posait les bases de la reconnaissance de formes géométriques à partir d'images. Dans les années 1980, David Marr proposa une théorie influente de la vision computationnelle, décomposant le processus en trois niveaux : le croquis primal (détection de contours), le croquis 2.5D (orientation des surfaces) et la représentation 3D complète.

Les années 1990 virent l'essor des méthodes basées sur les caractéristiques locales, notamment SIFT (*Scale-Invariant Feature Transform*) proposé par David Lowe en 1999. Ces descripteurs permettaient de reconnaître des objets indépendamment de leur échelle, rotation ou conditions d'illumination. En parallèle, les Support Vector Machines (SVM) offraient un cadre de classification performant pour ces représentations.

Le tournant décisif survint en 2012, lorsque le réseau AlexNet, conçu par Alex Krizhevsky, Ilya Sutskever et Geoffrey Hinton, remporta le challenge ImageNet avec une marge considérable. Ce résultat démontra la puissance des réseaux de neurones convolutifs (CNN) entraînés sur GPU, ouvrant la voie à l'ère moderne du deep learning appliqué à la vision.

TABLE 1.1 – Chronologie des avancées majeures en vision par ordinateur

Année	Événement	Impact
1966	Projet Summer Vision (MIT)	Première tentative formelle de vision artificielle
1980	Théorie de Marr	Cadre théorique pour la vision computationnelle
1998	LeNet-5 (LeCun)	Premier CNN performant pour la reconnaissance de chiffres
1999	SIFT (Lowe)	Descripteurs locaux robustes aux transformations
2012	AlexNet (Krizhevsky)	Révolution du deep learning en vision par ordinateur
2015	ResNet (He et al.)	Réseaux très profonds avec connexions résiduelles
2020	Vision Transformers	Architectures Transformer appliquées aux images

1.3 Tâches fondamentales

La vision par ordinateur englobe de nombreuses tâches, que l'on peut organiser par niveau croissant de complexité sémantique :

— **Classification d'images** : attribuer une étiquette (classe) à une image entière. Exemple : distinguer un chat d'un chien.

— **Détection d'objets** : localiser les objets dans une image par des boîtes englobantes (*bounding boxes*) et les classifier. Architectures clés : YOLO, Faster R-CNN, SSD.

— **Segmentation sémantique** : attribuer une classe à chaque pixel de l'image. Chaque pixel reçoit une étiquette de classe (route, piéton, véhicule, ciel...).

— **Segmentation d'instance** : distinguer chaque occurrence individuelle d'un objet. Deux personnes reçoivent des masques différents.

— **Estimation de pose** : déterminer la position et l'orientation d'un objet ou du corps humain (squelette articulaire).

— **Reconstruction 3D** : reconstruire la géométrie tridimensionnelle d'une scène à partir d'images 2D (stéréovision, *structure from motion*).

1.4 Enjeux et défis actuels

Malgré les progrès spectaculaires, de nombreux défis subsistent. La robustesse aux perturbations adversariales constitue un problème critique : de minuscules modifications imperceptibles à l'œil humain peuvent tromper complètement un réseau de neurones. Le manque de données étiquetées dans certains domaines spécifiques (imagerie médicale, imagerie satellitaire) reste un frein majeur.

L'interprétabilité des modèles profonds constitue également un enjeu crucial, particulièrement dans les domaines réglementés comme la santé et l'automobile. Comprendre pourquoi un réseau prend une décision donnée est essentiel pour instaurer la confiance. Ce sujet sera traité en détail dans les chapitres 9 et 10 de ce polycopié.

Enfin, l'efficacité computationnelle est un enjeu pratique : déployer des modèles profonds sur des dispositifs embarqués (smartphones, drones, véhicules autonomes) nécessite des techniques de compression, de distillation de connaissances ou d'architectures légères.

Point clé

La vision par ordinateur ne se réduit pas à la classification d'images. Elle englobe un spectre de tâches allant du traitement bas niveau (filtrage, détection de contours) à la compréhension sémantique de haut niveau (description textuelle de scènes, raisonnement visuel).

Chapitre 2

Représentation mathématique des images

2.1 Image numérique : définition formelle

Une image numérique en niveaux de gris peut être modélisée comme une fonction discrète définie sur un domaine rectangulaire. Plus précisément, une image est une application :

Définition — Image numérique

$$I : \{0, 1, \ldots, H-1\} \times \{0, 1, \ldots, W-1\} \longrightarrow \{0, 1, \ldots, 255\}$$

où H désigne la hauteur (nombre de lignes) et W la largeur (nombre de colonnes).

Chaque couple (i, j) désigne un **pixel** (*picture element*) et $I(i, j)$ représente son intensité lumineuse, codée généralement sur 8 bits (valeurs de 0 à 255).

En pratique, une image est stockée sous forme d'un **tenseur**, c'est-à-dire un tableau multi-dimensionnel. Pour une image en niveaux de gris, il s'agit d'une matrice de dimension $H \times W$. Pour une image couleur, on utilise un tenseur de dimension $H \times W \times C$, où C est le nombre de canaux (typiquement 3 pour RGB).

2.2 Espaces de couleur

L'espace **RGB** (*Red, Green, Blue*) est le plus utilisé en informatique. Chaque pixel est représenté par un triplet (R, G, B) où chaque composante varie de 0 à 255. Le mélange additif de ces trois primaires permet de représenter environ 16,7 millions de couleurs (2^{24}).

D'autres espaces sont utilisés selon les applications. L'espace **HSV** (*Hue, Saturation, Value*) sépare la teinte de la luminosité, facilitant certaines tâches de segmentation. L'espace **LAB** (CIE L*a*b*) est conçu pour être perceptuellement uniforme : une même distance euclidienne dans cet espace correspond à une même différence perçue par l'œil humain.

Table 2.1 – Principaux espaces de couleur

Espace	**Composantes**	**Usage typique**
RGB	Rouge, Vert, Bleu	Affichage, stockage standard
HSV	Teinte, Saturation, Valeur	Segmentation par couleur
LAB	Luminosité, a*, b*	Comparaison perceptuelle
YCbCr	Luminance, Chrominances	Compression vidéo (JPEG, MPEG)
Niveaux de gris	Intensité	Traitement classique, réduction de complexité

2.3 Représentation tensorielle

En apprentissage profond, les images sont manipulées comme des tenseurs. La convention de PyTorch place le nombre de canaux en première dimension : un tenseur d'image a la forme (C, H, W). Un **batch** (lot) d'images est un tenseur de dimension (N, C, H, W), où N est la taille du batch.

Il est important de **normaliser** les valeurs des pixels avant de les fournir à un réseau de neurones. La normalisation la plus courante consiste à diviser par 255 pour ramener les valeurs dans $[0, 1]$, puis à centrer et réduire chaque canal selon les statistiques du jeu de données d'entraînement :

$$\hat{x} = \frac{x - \mu}{\sigma}$$

Par exemple, pour ImageNet, les moyennes par canal sont $\mu = (0{,}485;\ 0{,}456;\ 0{,}406)$ et les écarts-types $\sigma = (0{,}229;\ 0{,}224;\ 0{,}225)$. Cette normalisation améliore la convergence de l'entraînement en centrant les activations autour de zéro.

2.4 Histogramme et statistiques d'une image

L'histogramme d'une image en niveaux de gris représente la distribution des intensités. Pour chaque valeur $k \in \{0, \ldots, 255\}$, on compte le nombre de pixels ayant cette intensité :

$$h(k) = \big|\{(i, j) : I(i, j) = k\}\big|$$

L'histogramme normalisé $p(k) = h(k)/(H \times W)$ représente une estimation de la densité de probabilité des intensités. L'**égalisation d'histogramme** est une transformation qui redistribue les intensités pour obtenir un histogramme approximativement uniforme, améliorant ainsi le contraste global de l'image.

La fonction de répartition cumulative (CDF) est définie par :

$$F(k) = \sum_{i=0}^{k} p(i)$$

L'égalisation consiste alors à appliquer la transformation $T(k) = \lfloor 255 \cdot F(k) \rfloor$, qui étale les intensités concentrées et compresse celles qui sont rares.

2.5 Notion de voisinage et connectivité

Le traitement d'image exploite largement la notion de **voisinage** d'un pixel. Le voisinage **4-connexe** du pixel (i, j) est l'ensemble $\{(i{-}1, j),\ (i{+}1, j),\ (i, j{-}1),\ (i, j{+}1)\}$, c'est-à-dire les quatre pixels partageant une arête avec le pixel central. Le voisinage **8-connexe** ajoute les quatre diagonales.

La notion de connectivité est fondamentale pour la segmentation : deux pixels appartiennent à la même **région connectée** s'il existe un chemin de pixels voisins satisfaisant un critère donné (même intensité, même classe, etc.). Les algorithmes d'étiquetage en composantes connexes (*connected components labeling*) permettent d'identifier ces régions efficacement.

Rappel mathématique

Un tenseur d'ordre n est une généralisation des matrices à n dimensions. Un scalaire est un tenseur d'ordre 0, un vecteur d'ordre 1, une matrice d'ordre 2. En deep learning, on manipule couramment des tenseurs d'ordre 4 (batch d'images) voire 5 (batch de vidéos).

Chapitre 3

Convolution discrète

3.1 Définition mathématique

La convolution discrète est l'opération centrale du traitement d'image et des réseaux convolutifs. Pour une image I et un noyau (*kernel*) K de taille $(2m+1) \times (2n+1)$, la convolution discrète 2D est définie par :

Définition — Convolution discrète 2D

$$(I * K)(i, j) = \sum_{p=-m}^{m} \sum_{q=-n}^{n} I(i-p,\, j-q) \cdot K(p, q)$$

En pratique, la plupart des bibliothèques de deep learning implémentent la **corrélation croisée** plutôt que la convolution mathématique au sens strict. La différence réside dans le signe des indices : la corrélation utilise $I(i+p, j+q)$ au lieu de $I(i-p, j-q)$. Pour un noyau symétrique, les deux opérations sont identiques. Pour les noyaux appris par rétropropagation, la distinction n'a pas d'importance pratique car le réseau apprend le noyau adapté.

3.2 Propriétés fondamentales

La convolution discrète possède plusieurs propriétés mathématiques essentielles :

- **Commutativité** : $I * K = K * I$. L'ordre des opérandes n'affecte pas le résultat.
- **Associativité** : $(I * K_1) * K_2 = I * (K_1 * K_2)$. On peut précalculer la convolution de deux noyaux.
- **Distributivité** : $I * (K_1 + K_2) = I * K_1 + I * K_2$.
- **Linéarité** : $(\alpha I) * K = \alpha(I * K)$. La convolution est une opération linéaire.
- **Équivariance par translation** : si l'on décale l'image, la sortie est décalée de la même manière. Cette propriété est cruciale pour la détection de motifs indépendamment de leur position.

3.3 Gestion des bords

Lorsqu'on applique un noyau de convolution aux pixels situés au bord de l'image, certains voisins requis se trouvent en dehors du domaine. Plusieurs stratégies de **padding** existent :

- **Zero padding** : les pixels extérieurs sont considérés comme valant 0. C'est la méthode la plus utilisée en deep learning.

— **Réplication** (*replicate*) : le pixel de bord est répété indéfiniment.
— **Réflexion** (*reflect*) : l'image est reflétée symétriquement par rapport au bord.
— **Périodique** (*wrap*) : l'image est considérée comme périodique (topologie torique).

Le choix du padding affecte la taille de la sortie. Avec un noyau de taille $k \times k$, un padding de p pixels et un pas (*stride*) de s, la dimension de sortie est :

$$O = \left\lfloor \frac{W - k + 2p}{s} \right\rfloor + 1$$

Pour conserver les dimensions spatiales (*same convolution*), on utilise $p = \lfloor k/2 \rfloor$ avec $s = 1$. Sans padding (*valid convolution*), la sortie est réduite de $(k - 1)$ pixels dans chaque dimension.

3.4 Stride et dilatation

Le **stride** (pas) détermine l'espacement entre les positions successives du noyau. Un stride de 2 réduit par deux les dimensions spatiales de la sortie, jouant un rôle similaire au pooling pour la réduction de dimensionnalité.

La **convolution dilatée** (*atrous convolution*) insère des zéros entre les éléments du noyau, augmentant ainsi le champ récepteur sans augmenter le nombre de paramètres. Un noyau 3×3 avec un taux de dilatation $d = 2$ couvre la même zone qu'un noyau 5×5 standard, avec seulement 9 paramètres au lieu de 25 :

$$\text{Champ récepteur effectif} = k + (k - 1)(d - 1) = d(k - 1) + 1$$

3.5 Complexité computationnelle

La convolution naïve d'une image $H \times W$ avec un noyau $k \times k$ a une complexité en $\mathcal{O}(H \times W \times k^2)$. Pour de grands noyaux, la convolution dans le domaine de Fourier est plus efficace : la transformée de Fourier discrète (FFT) transforme la convolution en multiplication point à point, avec une complexité en $\mathcal{O}(HW \log(HW))$.

En deep learning, les noyaux sont généralement petits (3×3 ou 5×5) et la convolution directe est optimisée par des bibliothèques spécialisées (cuDNN). L'approche **im2col** transforme la convolution en une multiplication matricielle dense, exploitant efficacement les instructions SIMD et les caches GPU.

Exercice

Calculez la dimension de sortie d'une convolution avec une image 32×32, un noyau 5×5, un padding de 2 et un stride de 1. Vérifiez ensuite avec un stride de 2.

Chapitre 4

Filtres et extraction de caractéristiques

4.1 Filtres de lissage

Le lissage (ou filtrage passe-bas) vise à réduire le bruit dans une image en atténuant les hautes fréquences. Le **filtre moyen** remplace chaque pixel par la moyenne de ses voisins. Pour un noyau 3×3, chaque élément vaut $1/9$. Ce filtre est simple mais estompe les contours.

Le **filtre gaussien** offre un meilleur compromis entre réduction du bruit et préservation des contours. Le noyau est défini par la distribution gaussienne 2D :

$$G(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right)$$

Le paramètre σ contrôle l'étendue du lissage : un σ élevé produit un lissage plus fort. Le filtre gaussien est **séparable** : un noyau 2D peut être décomposé en deux convolutions 1D successives (horizontale puis verticale), réduisant la complexité de $\mathcal{O}(k^2)$ à $\mathcal{O}(2k)$ par pixel.

Le **filtre médian**, quant à lui, est un filtre non linéaire qui remplace chaque pixel par la médiane de ses voisins. Il est particulièrement efficace contre le bruit impulsionnel (*salt-and-pepper noise*) car il préserve mieux les contours que le filtre moyen.

4.2 Détection de contours

Les contours correspondent aux zones de forte variation d'intensité. La détection de contours repose sur le calcul du **gradient** de l'image. Le gradient d'une image I en un pixel (i, j) est le vecteur :

$$\nabla I(i, j) = \left(\frac{\partial I}{\partial x}, \frac{\partial I}{\partial y}\right)$$

La magnitude du gradient $|\nabla I| = \sqrt{\left(\frac{\partial I}{\partial x}\right)^2 + \left(\frac{\partial I}{\partial y}\right)^2}$ indique la force du contour, tandis que l'angle $\theta = \arctan\left(\frac{\partial I/\partial y}{\partial I/\partial x}\right)$ donne son orientation.

L'opérateur de **Sobel** est le plus utilisé pour approximer les dérivées partielles. Il combine un lissage gaussien avec une différenciation :

$$G_x = \begin{pmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{pmatrix}, \qquad G_y = \begin{pmatrix} +1 & +2 & +1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{pmatrix}$$

L'opérateur de **Canny** représente l'état de l'art pour la détection de contours en traitement d'image classique. Il procède en quatre étapes : lissage gaussien, calcul du gradient (Sobel), suppression des non-maxima (amincissement des contours), et seuillage par hystérésis (double seuil pour relier les contours forts aux contours faibles).

4.3 Détection de coins et points d'intérêt

Les coins sont des points où le gradient change significativement dans au moins deux directions. Le **détecteur de Harris** repose sur la matrice d'auto-corrélation $\mathbf{M}$, construite à partir des dérivées partielles :

$$\mathbf{M} = \sum_{(x,y)\in\mathcal{W}} w(x,y) \begin{pmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{pmatrix}$$

où w est une fenêtre de pondération (typiquement gaussienne). La réponse de Harris est définie par :

$$R = \det(\mathbf{M}) - k \cdot \mathrm{trace}(\mathbf{M})^2 \qquad \text{avec } k \approx 0{,}04 \text{ à } 0{,}06$$

Un point est considéré comme un coin si R dépasse un seuil.

4.4 Descripteurs locaux : de SIFT aux features apprises

SIFT (*Scale-Invariant Feature Transform*) est un pipeline complet de détection et description de points d'intérêt. Il détecte les points dans l'espace des échelles (Difference of Gaussians), calcule l'orientation dominante, puis construit un descripteur de 128 dimensions basé sur les histogrammes d'orientation du gradient dans le voisinage.

HOG (*Histogram of Oriented Gradients*) calcule des histogrammes d'orientation du gradient sur des cellules régulières de l'image, normalisés par blocs. Ce descripteur a été longtemps l'état de l'art pour la détection de piétons (Dalal et Triggs, 2005).

L'émergence du deep learning a rendu ces descripteurs manuels largement obsolètes pour la plupart des applications. Les premières couches d'un CNN apprennent automatiquement des filtres similaires aux opérateurs de Sobel et Gabor, tandis que les couches profondes extraient des caractéristiques de plus en plus abstraites et sémantiques.

Transition

Les filtres classiques (Sobel, Gabor, etc.) sont prédéfinis par le concepteur. L'idée fondatrice des CNN est de laisser le réseau **apprendre** ses propres filtres, optimisés pour la tâche à résoudre.

Chapitre 5

Architecture des réseaux de neurones convolutifs

5.1 Le neurone artificiel

Un neurone artificiel calcule une somme pondérée de ses entrées, ajoute un biais, puis applique une fonction d'activation non linéaire :

$$y = f\left(\boldsymbol{w}^\top \boldsymbol{x} + b\right) = f\left(\sum_{i=1}^{n} w_i x_i + b\right)$$

où $\boldsymbol{x} \in \mathbb{R}^n$ est le vecteur d'entrée, $\boldsymbol{w} \in \mathbb{R}^n$ le vecteur de poids, $b \in \mathbb{R}$ le biais, et f la fonction d'activation. Sans non-linéarité, un empilement de couches linéaires se réduirait à une seule transformation affine.

5.2 Fonctions d'activation

Table 5.1 – Principales fonctions d'activation

Fonction	**Définition**	**Avantage**	**Inconvénient**
Sigmoïde	$\sigma(x) = \frac{1}{1+e^{-x}}$	Sortie dans $]0, 1[$	*Vanishing gradient*, sortie non centrée
Tanh	$\tanh(x)$	Centrée sur 0	*Vanishing gradient* aux extrémités
ReLU	$\max(0, x)$	Rapide, pas de vanishing pour $x > 0$	*Dying ReLU* (gradient nul si $x < 0$)
Leaky ReLU	$\max(\alpha x, x)$	Pas de neurones morts	Hyperparamètre α
GELU	$x \cdot \Phi(x)$	Lisse, bonnes perf. empiriques	Plus coûteux à calculer

ReLU (*Rectified Linear Unit*) est la fonction d'activation standard des réseaux profonds depuis 2010. Sa simplicité ($\max(0, x)$) permet un calcul très rapide et son gradient constant de 1 pour les valeurs positives atténue le problème du *vanishing gradient*. Toutefois, les neurones dont l'entrée reste négative ne reçoivent plus de gradient (*dying ReLU*), ce qui peut être atténué par les variantes Leaky ReLU ou ELU.

5.3 Couche convolutive

La couche convolutive est le composant fondamental d'un CNN. Elle applique un ensemble de noyaux apprenables à l'entrée pour produire des **cartes de caractéristiques** (*feature maps*). Si l'entrée a C_{in} canaux et qu'on utilise C_{out} noyaux de taille $k \times k$, le nombre de paramètres est :

$$P = C_{\text{out}} \times (C_{\text{in}} \times k \times k + 1)$$

Le « +1 » correspond au biais par filtre. Chaque noyau est appliqué sur l'ensemble des canaux d'entrée (convolution 3D), produisant une unique carte de caractéristiques en sortie. L'empilement de C_{out} noyaux produit C_{out} cartes.

Le **partage de poids** est la propriété clé des CNN : le même noyau est appliqué à toutes les positions spatiales de l'entrée. Cela réduit drastiquement le nombre de paramètres comparé à une couche entièrement connectée (*fully connected*) et confère l'équivariance par translation.

5.4 Couche de pooling

Le **pooling** réduit les dimensions spatiales des cartes de caractéristiques, diminuant la quantité de calcul et introduisant une forme d'invariance locale à la translation. Le **max pooling** retient la valeur maximale dans chaque fenêtre, tandis que l'**average pooling** calcule la moyenne.

Le max pooling 2×2 avec stride 2 divise par deux chaque dimension spatiale. Le **global average pooling** (GAP) calcule la moyenne sur l'ensemble de la carte de caractéristiques, produisant un scalaire par canal. Il est souvent utilisé juste avant la couche de classification, remplaçant avantageusement les couches fully connected coûteuses.

5.5 Architecture canonique d'un CNN

Un CNN classique suit le schéma suivant : une succession de blocs convolutifs (convolution + activation + pooling) suivis d'une ou plusieurs couches entièrement connectées. Les premières couches détectent des caractéristiques bas niveau (contours, textures), tandis que les couches profondes captent des motifs de plus en plus abstraits.

Listing 5.1 – Architecture d'un CNN simple pour CIFAR-10

```
Entree : 32 x 32 x 3
-> Conv(3x3, 32 filtres) + ReLU -> 32 x 32 x 32
-> MaxPool(2x2) -> 16 x 16 x 32
-> Conv(3x3, 64 filtres) + ReLU -> 16 x 16 x 64
-> MaxPool(2x2) -> 8 x 8 x 64
-> Conv(3x3, 128 filtres) + ReLU -> 8 x 8 x 128
-> Global Average Pooling -> 128
-> FC(128, 10) + Softmax -> 10
```

La couche **Softmax** transforme les logits (scores bruts) en probabilités de classe. Pour un vecteur $\boldsymbol{z}$ de dimension K :

$$\text{softmax}(\boldsymbol{z})_i = \frac{\exp(z_i)}{\sum_{j=1}^{K} \exp(z_j)}$$

Exercice

Pour le CNN ci-dessus, calculez le nombre total de paramètres de chaque couche. Vérifiez que la couche FC contient $128 \times 10 + 10 = 1290$ paramètres.

Chapitre 6

Optimisation et apprentissage

6.1 Fonction de perte

La **fonction de perte** (*loss function*) quantifie l'écart entre la prédiction du modèle et la vérité terrain. En classification, la perte d'**entropie croisée** (*cross-entropy*) est la plus utilisée :

$$\mathcal{L} = -\sum_{i=1}^{K} y_i \log(\hat{y}_i)$$

où $\boldsymbol{y}$ est le vecteur one-hot de la classe vraie et $\hat{\boldsymbol{y}}$ le vecteur de probabilités prédites (sortie du Softmax). Pour la classification binaire, cela se simplifie en :

$$\mathcal{L} = -\big[y \log(\hat{y}) + (1-y)\log(1-\hat{y})\big]$$

En régression, la perte **MSE** (*Mean Squared Error*) est standard : $\mathcal{L} = \frac{1}{N}\sum_i (y_i - \hat{y}_i)^2$. La perte **MAE** est plus robuste aux valeurs aberrantes mais non différentiable en zéro. La perte de **Huber** combine les avantages des deux : quadratique près de zéro, linéaire au-delà d'un seuil δ.

6.2 Descente de gradient stochastique (SGD)

L'entraînement d'un réseau de neurones consiste à minimiser la perte moyenne sur le jeu d'entraînement en ajustant les paramètres $\boldsymbol{\theta}$:

$$\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}} \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}(f(\boldsymbol{x}_i; \boldsymbol{\theta}),\, y_i)$$

La descente de gradient met à jour les paramètres dans la direction opposée au gradient :

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \cdot \nabla_{\boldsymbol{\theta}} \mathcal{L}$$

où η est le **taux d'apprentissage** (*learning rate*). En pratique, calculer le gradient exact sur tout le jeu de données est coûteux. La SGD utilise un **mini-batch** (sous-ensemble aléatoire) à chaque itération, fournissant une estimation bruitée mais non biaisée du gradient. Ce bruit est bénéfique : il aide à échapper aux minima locaux.

6.3 Algorithmes d'optimisation avancés

6.3.1 SGD avec momentum

Le momentum accélère la convergence en accumulant une moyenne exponentielle des gradients passés :

$$\boldsymbol{v} \leftarrow \beta \boldsymbol{v} + \nabla \mathcal{L}(\boldsymbol{\theta}) \tag{6.1}$$

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \boldsymbol{v} \tag{6.2}$$

Le coefficient β (typiquement 0,9) contrôle l'inertie. Le momentum atténue les oscillations dans les directions à forte courbure et accélère la progression dans les directions à faible courbure.

6.3.2 Adam

Adam (*Adaptive Moment Estimation*) combine le momentum avec une adaptation du taux d'apprentissage par paramètre. Il maintient deux estimations :

$$\boldsymbol{m} \leftarrow \beta_1 \boldsymbol{m} + (1 - \beta_1) \nabla \mathcal{L} \qquad \text{(premier moment, moyenne)} \tag{6.3}$$

$$\boldsymbol{v} \leftarrow \beta_2 \boldsymbol{v} + (1 - \beta_2)(\nabla \mathcal{L})^2 \qquad \text{(second moment, variance)} \tag{6.4}$$

Après correction du biais (division par $1 - \beta^t$), la mise à jour est :

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \cdot \frac{\hat{\boldsymbol{m}}}{\sqrt{\hat{\boldsymbol{v}}} + \varepsilon}$$

Les valeurs par défaut ($\beta_1 = 0{,}9$, $\beta_2 = 0{,}999$, $\varepsilon = 10^{-8}$) fonctionnent bien dans la majorité des cas. Adam converge plus rapidement que SGD en début d'entraînement, mais SGD avec momentum et scheduling généralise souvent mieux pour les tâches de classification d'images.

6.4 Rétropropagation du gradient

La **rétropropagation** (*backpropagation*) est l'application de la règle de la chaîne pour calculer le gradient de la perte par rapport à chaque paramètre du réseau. Pour une composition de fonctions $f = f_L \circ \cdots \circ f_1$:

$$\frac{\partial \mathcal{L}}{\partial \theta_k} = \frac{\partial \mathcal{L}}{\partial a_L} \cdot \frac{\partial a_L}{\partial a_{L-1}} \cdots \frac{\partial a_{k+1}}{\partial a_k} \cdot \frac{\partial a_k}{\partial \theta_k}$$

La rétropropagation procède en deux passes : la **passe avant** (*forward pass*) calcule les activations couche par couche, puis la **passe arrière** (*backward pass*) propage le gradient de la perte vers les entrées, accumulant les gradients des paramètres au passage. L'autograd de PyTorch automatise entièrement ce processus grâce à un graphe de calcul dynamique.

6.5 Planification du taux d'apprentissage

Le learning rate est l'hyperparamètre le plus influent. Plusieurs stratégies de scheduling existent :

- **Step decay** : diviser le LR par un facteur (ex. 10) tous les N époques.
- **Cosine annealing** : décroissance suivant un cosinus : $\eta(t) = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})(1 + \cos(\pi t/T))$.
- **Warm-up** : augmenter progressivement le LR pendant les premières itérations, puis appliquer le scheduling principal. Essentiel pour les grands batch sizes.
- **One-cycle policy** : un cycle unique d'augmentation puis diminution du LR.

Conseil pratique

Commencez toujours par un **LR finder** (balayage logarithmique du taux d'apprentissage sur quelques batches) pour identifier la plage optimale. Le LR optimal se situe généralement là où la perte décroît le plus rapidement, un ordre de grandeur avant la divergence.

Chapitre 7

Régularisation

7.1 Le problème du surapprentissage

Le **surapprentissage** (*overfitting*) survient lorsqu'un modèle mémorise les données d'entraînement au lieu d'en extraire les règles générales. Un réseau profond avec des millions de paramètres possède une capacité de mémorisation suffisante pour apprendre parfaitement n'importe quel jeu d'entraînement, y compris un jeu à étiquettes aléatoires (Zhang et al., 2017).

Le diagnostic du surapprentissage repose sur la comparaison des courbes d'entraînement et de validation : si la perte d'entraînement continue de diminuer tandis que la perte de validation stagne ou augmente, le modèle surapprend. Le **compromis biais-variance** est au cœur de ce phénomène : un modèle trop simple (fort biais, faible variance) sous-apprend, tandis qu'un modèle trop complexe (faible biais, forte variance) surapprend.

7.2 Régularisation L2 (Weight Decay)

La régularisation L2 ajoute un terme de pénalité proportionnel au carré de la norme des poids à la fonction de perte :

$$\mathcal{L}_{\text{reg}} = \mathcal{L} + \frac{\lambda}{2} \sum_i w_i^2$$

Ce terme pousse les poids vers zéro, limitant la complexité effective du modèle. En termes de mise à jour, le *weight decay* revient à multiplier les poids par $(1 - \eta\lambda)$ avant chaque mise à jour. La valeur typique de λ est de l'ordre de 10^{-4} à 5×10^{-4}.

Attention

Avec Adam, la régularisation L2 classique et le *weight decay* découplé (**AdamW**) ne sont pas équivalents. AdamW applique le decay directement sur les poids, indépendamment de l'estimation adaptative du gradient.

7.3 Dropout

Le **Dropout** (Srivastava et al., 2014) désactive aléatoirement une fraction p des neurones à chaque itération d'entraînement. Formellement, pour chaque neurone, on échantillonne un masque binaire $m \sim \text{Bernoulli}(1 - p)$ et l'activation devient :

$$\hat{\boldsymbol{h}} = \frac{\boldsymbol{m} \odot \boldsymbol{h}}{1 - p}$$

La division par $(1-p)$ (*inverted dropout*) garantit que l'espérance des activations reste inchangée, évitant de modifier l'échelle lors de l'inférence (où le dropout est désactivé).

Le dropout peut être interprété comme un **ensemble implicite de sous-réseaux** : à chaque itération, un sous-réseau différent est entraîné, et l'inférence revient à moyenner les prédictions de tous les sous-réseaux possibles. Les valeurs typiques de p sont 0,5 pour les couches FC et 0,1 à 0,3 pour les couches convolutives.

7.4 Batch Normalization

La **Batch Normalization** (Ioffe et Szegedy, 2015) normalise les activations de chaque couche sur le mini-batch :

$$\hat{y} = \gamma \cdot \frac{x - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \varepsilon}} + \beta$$

où $\mu_{\mathcal{B}}$ et $\sigma_{\mathcal{B}}^2$ sont la moyenne et la variance du mini-batch, et γ, β sont des paramètres apprenables. La BatchNorm réduit le problème de *covariate shift* interne, stabilise l'entraînement, permet des learning rates plus élevés et agit comme un régularisateur léger.

Lors de l'inférence, les statistiques du batch sont remplacées par des moyennes mobiles accumulées pendant l'entraînement. Des alternatives existent : la **Layer Normalization** (utilisée dans les Transformers), la **Group Normalization** (indépendante de la taille du batch), et l'**Instance Normalization** (utilisée en transfert de style).

7.5 Augmentation de données

L'**augmentation de données** (*data augmentation*) est la technique de régularisation la plus efficace en vision par ordinateur. Elle consiste à appliquer des transformations aléatoires aux images d'entraînement pour augmenter artificiellement la taille et la diversité du jeu de données :

- **Transformations géométriques** : rotation, retournement horizontal/vertical, recadrage aléatoire (*random crop*), mise à l'échelle, déformation affine.
- **Transformations photométriques** : variation de luminosité, contraste, saturation, teinte, ajout de bruit gaussien.
- **Méthodes avancées** : **Cutout** (masquage d'une région rectangulaire), **Mixup** (interpolation de paires d'images et de labels), **CutMix** (remplacement d'une région par un patch d'une autre image).

RandAugment et **AutoAugment** automatisent la sélection des transformations et de leurs paramètres, atteignant des performances supérieures aux stratégies manuelles.

7.6 Early Stopping

L'**arrêt précoce** (*early stopping*) consiste à surveiller la performance sur le jeu de validation et à arrêter l'entraînement lorsque celle-ci cesse de s'améliorer. Un compteur de *patience* permet de tolérer un nombre fixe d'époques sans amélioration avant l'arrêt. Le modèle retenu est celui ayant obtenu la meilleure performance de validation, pas nécessairement le dernier.

Résumé

Les techniques de régularisation ne sont pas mutuellement exclusives. En pratique, on combine augmentation de données, BatchNorm, weight decay et dropout pour obtenir les meilleures performances de généralisation.

Chapitre 8

Architectures modernes

8.1 VGGNet (2014)

VGGNet (Simonyan et Zisserman) démontra que la profondeur du réseau est un facteur déterminant de la performance. L'architecture utilise exclusivement des noyaux 3×3 empilés : deux couches 3×3 successives offrent le même champ récepteur qu'un noyau 5×5, mais avec moins de paramètres et davantage de non-linéarités.

VGG-16 contient 16 couches de poids (13 convolutives + 3 FC) et environ 138 millions de paramètres. La majorité des paramètres se trouvent dans les couches FC finales. Malgré ses performances, VGGNet est considéré inefficace par rapport aux architectures plus récentes.

8.2 GoogLeNet / Inception (2014)

GoogLeNet introduit le **module Inception**, qui applique en parallèle des convolutions de différentes tailles (1×1, 3×3, 5×5) et un max pooling, puis concatène les résultats. Les convolutions 1×1 réduisent la dimensionnalité avant les convolutions plus larges, limitant le coût computationnel.

Avec seulement 5 millions de paramètres (contre 138M pour VGG-16), GoogLeNet atteint des performances supérieures, démontrant qu'une architecture bien conçue surpasse la force brute d'un réseau simplement plus large ou plus profond.

8.3 ResNet (2015)

ResNet (He et al.) est l'une des contributions les plus influentes de l'histoire du deep learning. Son innovation clé est la **connexion résiduelle** (*skip connection*) : au lieu d'apprendre une transformation directe $H(\boldsymbol{x})$, chaque bloc apprend le résidu $F(\boldsymbol{x}) = H(\boldsymbol{x}) - \boldsymbol{x}$, et la sortie est :

Bloc résiduel

$$\boldsymbol{y} = F(\boldsymbol{x}) + \boldsymbol{x}$$

où F représente les couches du bloc (convolutions + BN + ReLU).

Cette formulation résout le problème de la **dégradation** : lorsqu'on ajoute des couches à un réseau standard, la performance peut paradoxalement diminuer sur le jeu d'entraînement. Les connexions résiduelles facilitent l'optimisation en fournissant un chemin direct pour le gradient, permettant l'entraînement de réseaux de 50, 101 voire 152 couches.

Le bloc résiduel **bottleneck** utilisé dans ResNet-50+ utilise trois convolutions : 1×1 (réduction), 3×3 (traitement), 1×1 (expansion). Cela réduit considérablement le nombre de

paramètres tout en maintenant le champ récepteur.

8.4 DenseNet (2017)

DenseNet (Huang et al.) pousse la logique des skip connections à l'extrême : chaque couche reçoit en entrée la concaténation des sorties de *toutes* les couches précédentes au sein du même bloc dense. Si chaque couche produit k feature maps (le **taux de croissance**), la couche ℓ reçoit $k_0 + (\ell - 1) \times k$ canaux en entrée.

DenseNet favorise la réutilisation des features, réduit le nombre de paramètres par rapport à ResNet pour des performances comparables, et atténue le problème du vanishing gradient grâce aux connexions directes multiples.

8.5 EfficientNet (2019)

EfficientNet (Tan et Le) propose une méthode systématique de mise à l'échelle des réseaux. Au lieu d'augmenter arbitrairement la profondeur, la largeur ou la résolution, le **compound scaling** les ajuste simultanément selon un ratio fixe optimisé par recherche d'architecture (NAS).

EfficientNet-B0 est le réseau de base, et les variantes B1 à B7 sont obtenues par scaling progressif. EfficientNet-B7 surpasse ResNet et DenseNet tout en étant significativement plus compact.

8.6 Vision Transformers (2020)

Le **Vision Transformer** (ViT) de Dosovitskiy et al. applique l'architecture Transformer, initialement conçue pour le NLP, directement aux images. L'image est découpée en **patches** (par exemple 16×16), chaque patch est projeté linéairement en un vecteur d'embedding, et la séquence de patches est traitée par un Transformer standard.

Le mécanisme d'**attention multi-tête** (*multi-head self-attention*) permet à chaque patch de « regarder » tous les autres patches de l'image, capturant des dépendances globales dès la première couche. Cela contraste avec les CNN, où le champ récepteur s'élargit progressivement avec la profondeur.

ViT nécessite de très grands jeux de données pour surpasser les CNN (typiquement JFT-300M ou ImageNet-21k), mais domine clairement lorsque suffisamment de données sont disponibles. Des variantes comme **DeiT** et **Swin Transformer** améliorent l'efficacité en introduisant respectivement la distillation et une attention locale hiérarchique.

Table 8.1 – Comparaison des architectures majeures sur ImageNet

Architecture	**Année**	**Param.**	**Top-1**	**Innovation clé**
VGG-16	2014	138M	71,5%	Empilements de 3×3
GoogLeNet	2014	5M	74,8%	Module Inception
ResNet-50	2015	25M	76,1%	Connexions résiduelles
DenseNet-121	2017	8M	74,4%	Connexions denses
EfficientNet-B0	2019	5,3M	77,1%	Compound scaling
ViT-B/16	2020	86M	77,9%*	Self-attention sur patches

* Performance après fine-tuning depuis ImageNet-21k.

Tendance actuelle

Les architectures hybrides CNN-Transformer combinent les avantages des deux approches : les convolutions pour les features locales bas niveau, l'attention pour les dépendances globales. ConvNeXt (2022) montre par ailleurs que les CNN, modernisés avec les techniques des Transformers, restent compétitifs.

Chapitre 9

Explainable AI (XAI)

9.1 Pourquoi l'explicabilité ?

Les réseaux de neurones profonds sont souvent qualifiés de « boîtes noires » : ils atteignent des performances remarquables mais n'offrent aucune explication transparente de leur raisonnement. Dans des domaines critiques comme la médecine, la justice ou la conduite autonome, déployer un modèle dont on ne comprend pas les décisions pose des problèmes éthiques, légaux et pratiques.

Le règlement européen **RGPD** et l'**AI Act** introduisent un droit à l'explication pour les décisions automatisées affectant les individus. L'explicabilité est également essentielle pour le débogage des modèles, la détection de biais, et l'établissement de la confiance des utilisateurs.

9.2 Taxonomie des méthodes d'explicabilité

Les méthodes d'explicabilité se classent selon plusieurs axes :

— **Intrinsèque vs post-hoc** : un modèle intrinsèquement interprétable (arbre de décision, régression linéaire) s'explique par sa structure, tandis que les méthodes post-hoc analysent un modèle déjà entraîné.

— **Globale vs locale** : une explication globale décrit le comportement général du modèle, une explication locale justifie une prédiction individuelle.

— **Model-agnostic vs model-specific** : les méthodes agnostiques fonctionnent avec n'importe quel modèle (LIME, SHAP), les méthodes spécifiques exploitent la structure interne (Grad-CAM pour les CNN).

9.3 LIME (*Local Interpretable Model-agnostic Explanations*)

LIME (Ribeiro et al., 2016) explique une prédiction individuelle en approximant localement le modèle complexe par un modèle interprétable (typiquement une régression linéaire). Pour une image, l'algorithme procède ainsi :

1. Segmenter l'image en super-pixels (régions cohérentes).
2. Générer des versions perturbées en masquant aléatoirement des super-pixels.
3. Obtenir la prédiction du modèle pour chaque version perturbée.
4. Entraîner un modèle linéaire sur ces exemples, pondérés par leur proximité à l'image originale.
5. Les coefficients du modèle linéaire indiquent l'importance de chaque super-pixel.

9.4 SHAP (*SHapley Additive exPlanations*)

SHAP (Lundberg et Lee, 2017) repose sur la théorie des jeux coopératifs et les **valeurs de Shapley**. La valeur de Shapley d'une caractéristique mesure sa contribution marginale moyenne à la prédiction, en considérant toutes les coalitions possibles :

$$\phi_i = \sum_{S \subseteq \mathcal{N} \setminus \{i\}} \frac{|S|!\,(n - |S| - 1)!}{n!} \Big[f(S \cup \{i\}) - f(S) \Big]$$

SHAP unifie plusieurs méthodes d'explicabilité (LIME, DeepLIFT, valeurs de Shapley classiques) sous un même cadre théorique. Ses propriétés désirables incluent l'**efficacité locale** (les contributions s'additionnent à la prédiction), la **symétrie** et la **monotonie**.

9.5 Méthodes basées sur le gradient

Les méthodes basées sur le gradient exploitent les gradients du modèle pour identifier les pixels les plus influents. La méthode **Vanilla Gradient** calcule simplement $\partial y_c / \partial \boldsymbol{x}$, le gradient de la sortie de la classe c par rapport à l'image d'entrée. Les cartes de saliency résultantes sont souvent bruitées.

Guided Backpropagation (Springenberg et al., 2015) modifie la règle de rétropropagation à travers les ReLU pour ne propager que les gradients positifs dans les deux sens, produisant des visualisations plus nettes.

SmoothGrad (Smilkov et al., 2017) moyenne les gradients sur N versions bruitées de l'image, réduisant le bruit haute fréquence des saliency maps. **Integrated Gradients** (Sundararajan et al., 2017) intègre le gradient le long d'un chemin reliant une image de référence (baseline, typiquement noire) à l'image d'entrée, satisfaisant des axiomes d'attribution désirables.

Attention

Les méthodes d'explicabilité ne sont pas infaillibles. Adebayo et al. (2018) ont montré que certaines méthodes de saliency produisent des cartes similaires même lorsque les poids du réseau sont randomisés, mettant en doute leur fidélité au modèle.

Chapitre 10

Grad-CAM

10.1 Principe général

Grad-CAM (*Gradient-weighted Class Activation Mapping*), proposé par Selvaraju et al. en 2017, est une méthode d'explicabilité visuelle qui produit une **carte de chaleur** (*heatmap*) mettant en évidence les régions de l'image les plus déterminantes pour la prédiction d'une classe donnée.

L'idée est simple et élégante : on utilise les gradients de la classe cible circulant vers la dernière couche convolutive pour pondérer les cartes d'activation. Les feature maps de la dernière couche convolutive encodent une information sémantique riche tout en conservant une résolution spatiale, ce qui en fait le choix naturel pour la localisation.

10.2 Formulation mathématique

Soit y^c le score (logit) de la classe c avant le Softmax, et A^k la k-ième feature map de la dernière couche convolutive de dimensions $H' \times W'$. Le poids d'importance du canal k pour la classe c est :

Poids Grad-CAM

$$\alpha_k^c = \underbrace{\frac{1}{H'W'} \sum_i \sum_j}_{\text{global average pooling}} \frac{\partial y^c}{\partial A_{ij}^k}$$

La carte Grad-CAM est ensuite :

$$L_{\text{Grad-CAM}}^c = \text{ReLU}\left(\sum_k \alpha_k^c \cdot A^k \right)$$

Le ReLU ne conserve que les influences positives (régions qui contribuent positivement à la classe d'intérêt). La carte résultante est une matrice $H' \times W'$, redimensionnée à la taille de l'image d'entrée par interpolation bilinéaire pour la visualisation.

10.3 Algorithme détaillé

1. **Passe avant** (*forward pass*) : propager l'image dans le réseau et enregistrer les activations de la couche convolutive cible.
2. Sélectionner la classe c d'intérêt (classe prédite ou toute autre classe).

3. **Rétropropagation** (*backward pass*) : calculer les gradients de y^c par rapport aux activations de la couche cible.
4. Calculer les poids α_k^c par global average pooling des gradients.
5. Calculer la combinaison linéaire pondérée des feature maps, suivie d'un ReLU.
6. Redimensionner la carte à la taille de l'image et superposer la heatmap.

10.4 Variantes

Grad-CAM++ (Chattopadhay et al., 2018) utilise une combinaison pondérée non uniforme des gradients positifs au lieu de la simple moyenne, améliorant la localisation lorsque plusieurs instances de la même classe sont présentes dans l'image.

Score-CAM (Wang et al., 2020) élimine complètement la dépendance aux gradients : chaque feature map est utilisée comme masque sur l'image d'entrée, et le poids de chaque canal est déterminé par l'augmentation du score de confiance lorsque la région est présente. Cette méthode est plus stable mais plus coûteuse.

Ablation-CAM génère la carte d'importance en retirant séquentiellement chaque feature map et en mesurant la baisse du score de classe. **Layer-CAM** (Jiang et al., 2021) étend Grad-CAM à toutes les couches convolutives, permettant des visualisations à différents niveaux de granularité.

10.5 Interprétation et limites

Grad-CAM révèle quelles régions spatiales le modèle « regarde » pour prendre sa décision, mais ne dit pas exactement quel motif ou texture est détecté dans ces régions. La résolution est limitée par celle de la dernière couche convolutive (7×7 pour ResNet-50), ce qui donne des cartes grossières.

Grad-CAM peut également révéler des **biais indésirables** : si un modèle de classification d'oiseaux se concentre sur l'arrière-plan (eau pour les oiseaux aquatiques, forêt pour les pics) plutôt que sur l'animal lui-même, Grad-CAM le rend visible. Cette capacité de débogage est l'un de ses usages les plus précieux.

Exercice

Implémentez Grad-CAM pour un ResNet-50 préentraîné sur ImageNet. Visualisez les heatmaps pour différentes classes et comparez les régions mises en évidence. Que se passe-t-il si vous ciblez une classe incorrecte ?

Chapitre 11

Implémentation PyTorch

11.1 Introduction à PyTorch

PyTorch est un framework open-source de calcul tensoriel et d'apprentissage automatique développé par Meta AI. Il se distingue par son graphe de calcul dynamique (*define-by-run*), permettant de modifier l'architecture du réseau à la volée, et par son interface Pythonique intuitive.

Les trois piliers de PyTorch sont : les **tenseurs** (`torch.Tensor`), analogues aux arrays NumPy mais exécutables sur GPU ; l'**autograd**, qui enregistre automatiquement les opérations pour la différentiation automatique ; et le module **nn**, qui fournit des couches et modèles prêts à l'emploi.

11.2 Tenseurs et opérations de base

Listing 11.1 – Opérations tensorielle de base en PyTorch

```
import torch

# Creation de tenseurs
x = torch.randn(3, 224, 224)          # Image aleatoire (C, H, W)
batch = torch.randn(16, 3, 224, 224) # Batch de 16 images

# Transfert GPU
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu'
    )
x = x.to(device)

# Operations tensorielles
y = x.view(3, -1)         # Reshape (3, 50176)
z = x.permute(1, 2, 0)    # (H, W, C) pour affichage
norm = x.norm()            # Norme L2
```

11.3 Définition d'un CNN avec `nn.Module`

La classe `nn.Module` est la brique de base pour définir des réseaux de neurones en PyTorch. Chaque module définit une méthode `forward()` qui spécifie le calcul à effectuer.

Listing 11.2 – Définition d'un CNN simple

```
import torch.nn as nn
```

```
import torch.nn.functional as F

class SimpleCNN(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1)
        self.bn1 = nn.BatchNorm2d(32)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.bn2 = nn.BatchNorm2d(64)
        self.conv3 = nn.Conv2d(64, 128, kernel_size=3, padding=1)
        self.bn3 = nn.BatchNorm2d(128)
        self.pool = nn.MaxPool2d(2, 2)
        self.gap = nn.AdaptiveAvgPool2d(1)
        self.fc = nn.Linear(128, num_classes)
        self.dropout = nn.Dropout(0.5)

    def forward(self, x):
        x = self.pool(F.relu(self.bn1(self.conv1(x))))
        x = self.pool(F.relu(self.bn2(self.conv2(x))))
        x = F.relu(self.bn3(self.conv3(x)))
        x = self.gap(x).view(x.size(0), -1)
        x = self.dropout(x)
        return self.fc(x)
```

11.4 Boucle d'entraînement

La boucle d'entraînement standard en PyTorch suit le schéma : forward pass, calcul de la perte, rétropropagation, mise à jour des poids.

Listing 11.3 – Boucle d'entraînement standard

```
from torch.optim import Adam
from torch.optim.lr_scheduler import CosineAnnealingLR

model = SimpleCNN(num_classes=10).to(device)
criterion = nn.CrossEntropyLoss()
optimizer = Adam(model.parameters(), lr=1e-3, weight_decay=1e-4)
scheduler = CosineAnnealingLR(optimizer, T_max=50)

for epoch in range(50):
    model.train()
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)

        optimizer.zero_grad()              # RAZ des gradients
        outputs = model(images)            # Forward pass
        loss = criterion(outputs, labels)  # Calcul de la perte
        loss.backward()                    # Retropropagation
        optimizer.step()                   # Mise a jour

    scheduler.step()
```

11.5 Chargement de données et augmentation

Listing 11.4 – Data loading avec augmentation

```
from torchvision import datasets, transforms
from torch.utils.data import DataLoader

train_transform = transforms.Compose([
    transforms.RandomHorizontalFlip(),
    transforms.RandomCrop(32, padding=4),
    transforms.ColorJitter(brightness=0.2, contrast=0.2),
    transforms.ToTensor(),
    transforms.Normalize((0.4914, 0.4822, 0.4465),
                         (0.2470, 0.2435, 0.2616)),
])

train_set = datasets.CIFAR10(root='./data', train=True,
                             download=True, transform=
                                train_transform)
train_loader = DataLoader(train_set, batch_size=128,
                          shuffle=True, num_workers=4)
```

11.6 Transfer learning

Le **transfer learning** consiste à réutiliser un modèle préentraîné sur un grand jeu de données (ImageNet) et à l'adapter à une tâche spécifique. Deux stratégies principales existent : le *feature extraction* (on gèle le backbone et on réentraîne uniquement la tête de classification) et le *fine-tuning* (on réentraîne l'ensemble du réseau avec un learning rate réduit).

Listing 11.5 – Transfer learning avec ResNet-50

```
from torchvision.models import resnet50, ResNet50_Weights

# Charger un ResNet-50 preentra\^ine
model = resnet50(weights=ResNet50_Weights.IMAGENET1K_V2)

# Geler le backbone
for param in model.parameters():
    param.requires_grad = False

# Remplacer la tete de classification
model.fc = nn.Sequential(
    nn.Dropout(0.3),
    nn.Linear(2048, num_classes)
)

# Seuls les parametres de model.fc seront mis a jour
optimizer = Adam(model.fc.parameters(), lr=1e-3)
```

11.7 Implémentation de Grad-CAM

Voici l'implémentation complète de Grad-CAM en PyTorch, utilisant les hooks pour intercepter les activations et gradients :

Listing 11.6 – Implémentation de Grad-CAM

```
class GradCAM:
```

```
    def __init__(self, model, target_layer):
        self.model = model.eval()
        self.activations = None
        self.gradients = None
        target_layer.register_forward_hook(
            lambda m, i, o: setattr(self, 'activations', o))
        target_layer.register_full_backward_hook(
            lambda m, gi, go: setattr(self, 'gradients', go[0]))

    def generate(self, input_tensor, class_idx=None):
        output = self.model(input_tensor)
        if class_idx is None:
            class_idx = output.argmax(dim=1).item()
        self.model.zero_grad()
        output[0, class_idx].backward()

        weights = self.gradients.mean(dim=[2, 3], keepdim=True)
        cam = (weights * self.activations).sum(dim=1, keepdim=True)
        cam = F.relu(cam)
        cam = F.interpolate(cam, input_tensor.shape[2:],
                            mode='bilinear', align_corners=False)
        cam = cam - cam.min()
        cam = cam / (cam.max() + 1e-8)
        return cam.squeeze().detach().cpu().numpy()
```

Conseil

Utilisez `torchinfo.summary(model, input_size=(1,3,224,224))` pour inspecter l'architecture, les dimensions de sortie et le nombre de paramètres de chaque couche.

Chapitre 12

Projet : application en imagerie médicale

12.1 Contexte : classification de radiographies thoraciques

Ce chapitre propose un projet intégrateur mobilisant l'ensemble des connaissances acquises dans ce cours. L'objectif est de construire un système de classification de radiographies thoraciques capable de détecter certaines pathologies (pneumonie, COVID-19, normal), en utilisant le transfer learning et l'explicabilité via Grad-CAM.

L'imagerie médicale représente un cas d'usage emblématique du deep learning en vision par ordinateur. Les enjeux sont considérables : le manque de radiologues dans de nombreux pays, la fatigue liée à l'examen de centaines d'images par jour, et la nécessité d'un diagnostic rapide en situation d'urgence.

12.2 Jeu de données

Plusieurs jeux de données publics sont disponibles pour la classification de radiographies thoraciques. Le jeu **Chest X-Ray Images** (Kermany et al., 2018) contient environ 5 800 radiographies réparties en trois classes : normal, pneumonie bactérienne, pneumonie virale. Le jeu **COVIDx** combine plusieurs sources pour inclure des cas de COVID-19.

Les défis spécifiques à l'imagerie médicale incluent le **déséquilibre des classes** (beaucoup plus de cas normaux que pathologiques), la variabilité des protocoles d'acquisition (différents appareils, doses, positionnements), et la petite taille des jeux de données comparés aux standards du deep learning.

Table 12.1 – Pipeline du projet

Étape	Action	Techniques
1	Prétraitement	Redimensionner à 224×224, normaliser
2	Augmentation	Rotation (± 15ř), flip, variation de contraste
3	Modèle	ResNet-50 préentraîné ImageNet
4	Entraînement	Fine-tuning progressif, LR scheduling
5	Évaluation	Matrice de confusion, ROC/AUC, sensibilité/spécificité
6	Explicabilité	Grad-CAM sur la dernière couche conv

12.3 Pipeline complet

12.3.1 Prétraitement des données

Listing 12.1 – Transformations pour l'imagerie médicale

```
from torchvision import transforms

# Transformations specifiques aux radiographies
medical_transform = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.Grayscale(num_output_channels=3),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406],
                         [0.229, 0.224, 0.225]),
])

# Augmentation pour l'entrainement
train_transform = transforms.Compose([
    transforms.Resize(256),
    transforms.RandomResizedCrop(224, scale=(0.8, 1.0)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(15),
    transforms.ColorJitter(brightness=0.3, contrast=0.3),
    transforms.Grayscale(num_output_channels=3),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406],
                         [0.229, 0.224, 0.225]),
])
```

12.3.2 Gestion du déséquilibre de classes

Pour gérer le déséquilibre des classes, plusieurs stratégies sont possibles. La plus simple consiste à utiliser un `WeightedRandomSampler` pour équilibrer les classes dans chaque batch. Une alternative est la **Focal Loss** (Lin et al., 2017), qui réduit le poids des exemples faciles et concentre l'entraînement sur les cas difficiles :

$$\mathrm{FL}(p) = -\alpha(1-p)^{\gamma}\log(p)$$

Le facteur $(1-p)^{\gamma}$ diminue la contribution des exemples bien classifiés (p élevé), tandis que α équilibre l'importance relative des classes. Les valeurs typiques sont $\gamma = 2$ et α proportionnel à l'inverse de la fréquence de chaque classe.

12.3.3 Fine-tuning progressif

Le fine-tuning progressif consiste à d'abord entraîner la tête de classification (couches FC) pendant quelques époques, puis à dégeler progressivement les couches du backbone en partant des couches les plus profondes.

Listing 12.2 – Fine-tuning progressif en trois phases

```
# Phase 1 : Entrainer la tete uniquement (5 epoques)
for param in model.parameters():
    param.requires_grad = False
for param in model.fc.parameters():
    param.requires_grad = True
train(model, epochs=5, lr=1e-3)
```

```
# Phase 2 : Degeler les 2 derniers blocs (10 epoques)
for param in model.layer4.parameters():
    param.requires_grad = True
for param in model.layer3.parameters():
    param.requires_grad = True
train(model, epochs=10, lr=1e-4)

# Phase 3 : Fine-tuning complet (10 epoques)
for param in model.parameters():
    param.requires_grad = True
train(model, epochs=10, lr=1e-5)
```

12.4 Métriques d'évaluation

En imagerie médicale, l'accuracy seule est insuffisante en raison du déséquilibre des classes. Les métriques essentielles sont :

— **Sensibilité (rappel)** : proportion de vrais positifs parmi les malades réels. En dépistage, une sensibilité élevée est critique pour ne pas manquer de cas pathologiques.

— **Spécificité** : proportion de vrais négatifs parmi les sujets sains.

— **AUC-ROC** : aire sous la courbe ROC, mesurant la capacité discriminante du modèle indépendamment du seuil de décision.

— **F1-score** : moyenne harmonique de la précision et du rappel, particulièrement informatif pour les classes minoritaires.

12.5 Explicabilité médicale avec Grad-CAM

L'explicabilité est cruciale en imagerie médicale. Grad-CAM permet de vérifier que le modèle se concentre sur les régions anatomiquement pertinentes. Pour une pneumonie, on s'attend à ce que la heatmap mette en évidence les zones pulmonaires affectées. Si le modèle se concentre sur les bords de l'image ou le texte incrusté, cela révèle un biais de raccourci (*shortcut learning*).

Listing 12.3 – Visualisation Grad-CAM pour une radiographie

```
import matplotlib.pyplot as plt
import numpy as np

grad_cam = GradCAM(model, model.layer4[-1])
cam = grad_cam.generate(input_tensor, class_idx=1)  # Pneumonie

fig, axes = plt.subplots(1, 3, figsize=(15, 5))
axes[0].imshow(original_image, cmap='gray')
axes[0].set_title('Radiographie originale')
axes[1].imshow(cam, cmap='jet')
axes[1].set_title('Carte Grad-CAM')
axes[2].imshow(original_image, cmap='gray')
axes[2].imshow(cam, cmap='jet', alpha=0.5)
axes[2].set_title('Superposition')
plt.savefig('gradcam_medical.png', dpi=150)
```

12.6 Considérations éthiques et réglementaires

Le déploiement d'un système d'aide au diagnostic médical implique des responsabilités considérables. Le modèle doit être validé cliniquement sur des cohortes indépendantes, représentatives de la population cible. Les biais liés à l'âge, au sexe, à l'équipement ou au centre d'acquisition doivent être identifiés et atténués.

Le marquage CE (en Europe) et l'approbation FDA (aux États-Unis) sont requis pour les dispositifs médicaux basés sur l'IA. Le cadre réglementaire exige une documentation exhaustive incluant les données d'entraînement, les performances par sous-groupe, les limitations connues, et les mécanismes de surveillance post-déploiement.

12.7 Cahier des charges du projet

Le projet se décompose en livrables progressifs :

1. **Rapport d'exploration des données** : statistiques descriptives, visualisation d'échantillons, analyse du déséquilibre des classes.
2. **Implémentation du pipeline de données** : Dataset, DataLoader, transformations et augmentation.
3. **Entraînement et évaluation** : courbes de perte et d'accuracy, matrice de confusion, rapport de classification.
4. **Analyse Grad-CAM** : visualisation des heatmaps sur des exemples représentatifs (vrais positifs, faux positifs, faux négatifs). Discussion de la cohérence médicale.
5. **Rapport final** : document structuré incluant la méthodologie, les résultats, l'analyse critique et les perspectives d'amélioration.

Livrable

Le projet doit être rendu sous forme d'un notebook Jupyter commenté, accompagné d'un rapport de 10 pages maximum. Le code doit être reproductible (seed fixé, `requirements.txt`). La soutenance inclut une démonstration en direct du pipeline.